

Subject :- Cyber Security

Practical 1

1.A) Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques

1.B) Design and implement algorithms to encrypt and decrypt messages using transposition techniques

1.A) Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques

Code:-

CLI:-

Practical 1A

----- Caesar Cipher -----

```
def caesar_encrypt(text, shift):
```

```
    result = ""
```

```
    for char in text:
```

```
        if char.isalpha():
```

```
            start = ord('A') if char.isupper() else ord('a')
```

```
            result += chr((ord(char) - start + shift) % 26 + start)
```

```
        else:
```

```
            result += char
```

```
    return result
```

```
def caesar_decrypt(text, shift):
```

```
return caesar_encrypt(text, -shift)
```

```
# ----- Playfair Cipher -----
```

```
def generate_key_table(key):
```

```
    key = key.upper().replace("J", "I")
```

```
    table = []
```

```
    used = set()
```

```
    for ch in key:
```

```
        if ch.isalpha() and ch not in used:
```

```
            table.append(ch)
```

```
            used.add(ch)
```

```
    for ch in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
```

```
        if ch not in used:
```

```
            table.append(ch)
```

```
            used.add(ch)
```

```
    matrix = []
```

```
    for i in range(0, 25, 5):
```

```
        matrix.append(table[i:i+5])
```

```
    return matrix
```

```
def find_position(table, ch):
```

```
    for i in range(5):
```

```
        for j in range(5):
```

```
            if table[i][j] == ch:
```

```
                return i, j
```

```
def prepare_text(text):
```

```
    text = text.upper().replace("J", "I")
```

```
    text = ".".join(c for c in text if c.isalpha())
```

```
    result = ""
```

```
    i = 0
```

```
    while i < len(text):
```

```
        a = text[i]
```

```
        if i + 1 < len(text):
```

```
            b = text[i + 1]
```

```
            if a == b:
```

```
                result += a + "X"
```

```
                i += 1
```

```
            else:
```

```
                result += a + b
```

```
                i += 2
```

else:

result += a + "X"

i += 1

return result

def playfair_encrypt(text, key):

table = generate_key_table(key)

text = prepare_text(text)

cipher = ""

for i in range(0, len(text), 2):

a = text[i]

b = text[i + 1]

r1, c1 = find_position(table, a)

r2, c2 = find_position(table, b)

if r1 == r2:

cipher += table[r1][(c1 + 1) % 5]

cipher += table[r2][(c2 + 1) % 5]

elif c1 == c2:

cipher += table[(r1 + 1) % 5][c1]

cipher += table[(r2 + 1) % 5][c2]

else:

 cipher += table[r1][c2]

 cipher += table[r2][c1]

return cipher

def playfair_decrypt(text, key):

 table = generate_key_table(key)

 plain = ""

 for i in range(0, len(text), 2):

 a = text[i]

 b = text[i + 1]

 r1, c1 = find_position(table, a)

 r2, c2 = find_position(table, b)

 if r1 == r2:

 plain += table[r1][(c1 - 1) % 5]

 plain += table[r2][(c2 - 1) % 5]

 elif c1 == c2:

 plain += table[(r1 - 1) % 5][c1]

 plain += table[(r2 - 1) % 5][c2]

else:

plain += table[r1][c2]

plain += table[r2][c1]

return plain

----- Main Menu -----

while True:

print("\n===== Substitution Cipher =====")

print("1. Caesar Encrypt")

print("2. Caesar Decrypt")

print("3. Playfair Encrypt")

print("4. Playfair Decrypt")

print("5. Exit")

choice = input("Enter Choice: ")

if choice == "1":

message = input("Enter Message: ")

key = int(input("Enter Shift Key: "))

print("Encrypted Message:", caesar_encrypt(message, key))

elif choice == "2":

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

```
message = input("Enter Cipher Text: ")
key = int(input("Enter Shift Key: "))
print("Decrypted Message:", caesar_decrypt(message, key))

elif choice == "3":
    key = input("Enter Playfair Key: ")
    message = input("Enter Message: ")
    print("Encrypted Message:", playfair_encrypt(message, key))

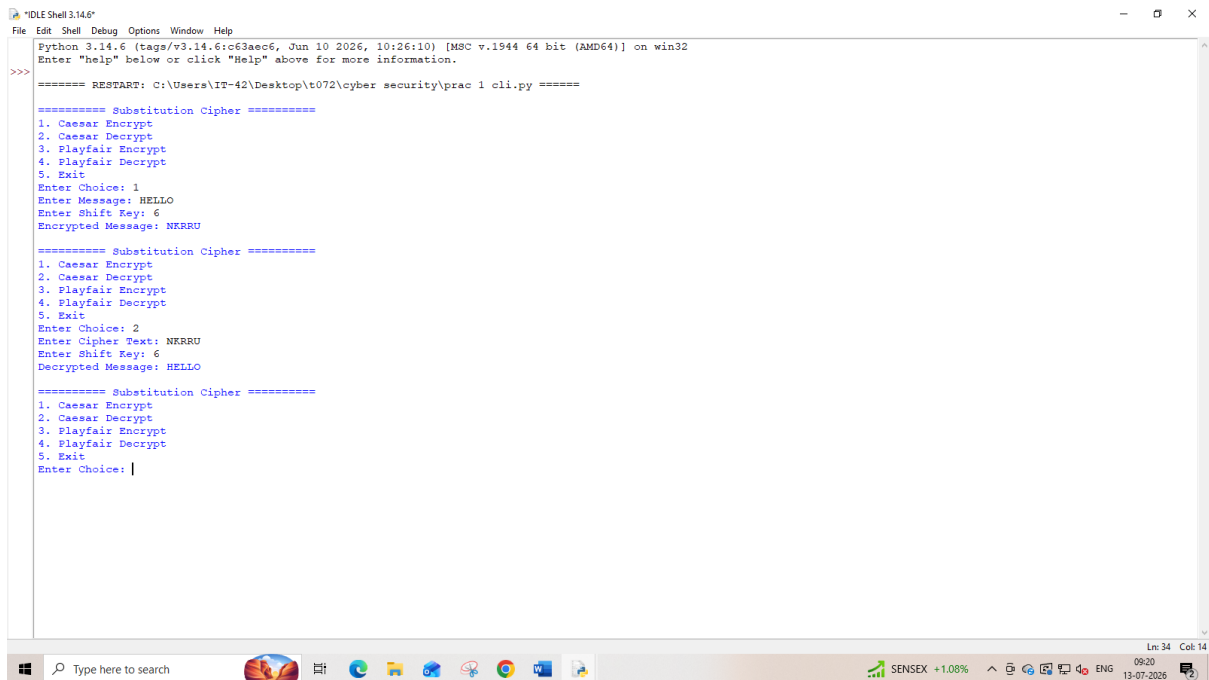
elif choice == "4":
    key = input("Enter Playfair Key: ")
    message = input("Enter Cipher Text: ")
    print("Decrypted Message:", playfair_decrypt(message, key))

elif choice == "5":
    print("Program Ended")
    break

else:
    print("Invalid Choice")
```

Output:-

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce



```
"IDLE Shell 3.14.6"
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:63aee6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\IT-42\Desktop\t072\cyber security\prac 1 cli.py =====

===== Substitution Cipher =====
1. Caesar Encrypt
2. Caesar Decrypt
3. Playfair Encrypt
4. Playfair Decrypt
5. Exit
Enter Choice: 1
Enter Message: HELLO
Enter Shift Key: 6
Encrypted Message: NKRRU

===== Substitution Cipher =====
1. Caesar Encrypt
2. Caesar Decrypt
3. Playfair Encrypt
4. Playfair Decrypt
5. Exit
Enter Choice: 2
Enter Cipher Text: NKRRU
Enter Shift Key: 6
Decrypted Message: HELLO

===== Substitution Cipher =====
1. Caesar Encrypt
2. Caesar Decrypt
3. Playfair Encrypt
4. Playfair Decrypt
5. Exit
Enter Choice: |
```

GUI:-

import tkinter as tk

from tkinter import ttk, messagebox

----- Caesar Cipher -----

def caesar_encrypt(text, shift):

 result = ""

 for ch in text:

 if ch.isalpha():

 start = ord('A') if ch.isupper() else ord('a')

 result += chr((ord(ch)-start+shift)%26+start)

 else:

 result += ch

 return result

def caesar_decrypt(text, shift):

YASH BHAGAT

T072

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

```
return caesar_encrypt(text, -shift)
```

```
# ----- Playfair Cipher -----
```

```
def generate_table(key):
```

```
    key = key.upper().replace("J","I")
```

```
    table=[]
```

```
    used=set()
```

```
    for ch in key:
```

```
        if ch.isalpha() and ch not in used:
```

```
            table.append(ch)
```

```
            used.add(ch)
```

```
    for ch in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
```

```
        if ch not in used:
```

```
            table.append(ch)
```

```
    return [table[i:i+5] for i in range(0,25,5)]
```

```
def find(table,ch):
```

```
    for i in range(5):
```

```
        for j in range(5):
```

```
            if table[i][j]==ch:
```

```
                return i,j
```

```
def prepare(text):
```

```
    text=text.upper().replace("J","I")
```

```
text=''.join(c for c in text if c.isalpha())
```

```
result=""
```

```
i=0
```

```
while i<len(text):
```

```
    a=text[i]
```

```
    if i+1<len(text):
```

```
        b=text[i+1]
```

```
        if a==b:
```

```
            result+=a+"X"
```

```
            i+=1
```

```
        else:
```

```
            result+=a+b
```

```
            i+=2
```

```
    else:
```

```
        result+=a+"X"
```

```
        i+=1
```

```
return result
```

```
def playfair_encrypt(text,key):
```

```
    table=generate_table(key)
```

```
    text=prepare(text)
```

```
    cipher=""
```

```
for i in range(0,len(text),2):  
    a,b=text[i],text[i+1]  
    r1,c1=find(table,a)  
    r2,c2=find(table,b)  
  
    if r1==r2:  
        cipher+=table[r1][(c1+1)%5]  
        cipher+=table[r2][(c2+1)%5]  
  
    elif c1==c2:  
        cipher+=table[(r1+1)%5][c1]  
        cipher+=table[(r2+1)%5][c2]  
  
    else:  
        cipher+=table[r1][c2]  
        cipher+=table[r2][c1]  
  
return cipher  
  
def playfair_decrypt(text,key):  
    table=generate_table(key)  
    plain=""  
  
    for i in range(0,len(text),2):  
        a,b=text[i],text[i+1]  
        r1,c1=find(table,a)  
        r2,c2=find(table,b)
```

```
if r1==r2:

    plain+=table[r1][(c1-1)%5]

    plain+=table[r2][(c2-1)%5]


elif c1==c2:

    plain+=table[(r1-1)%5][c1]

    plain+=table[(r2-1)%5][c2]


else:

    plain+=table[r1][c2]

    plain+=table[r2][c1]


return plain


# ----- GUI -----


def encrypt():

    algo=algorithm.get()

    if algo=="Caesar":

        output.config(text=caesar_encrypt(message.get(),int(key.get())))

    else:

        output.config(text=playfair_encrypt(message.get(),key.get()))


def decrypt():

    algo=algorithm.get()
```

```
if algo=="Caesar":
    output.config(text=caesar_decrypt(message.get(),int(key.get())))
else:
    output.config(text=playfair_decrypt(message.get(),key.get()))

root=tk.Tk()
root.title("Substitution Cipher")
root.geometry("400x350")

algorithm=tk.StringVar(value="Caesar")

tk.Label(root,text="Substitution Cipher",font=("Arial",16)).pack(pady=10)

tk.Radiobutton(root,text="Caesar Cipher",variable=algorithm,value="Caesar").pack()
tk.Radiobutton(root,text="Playfair Cipher",variable=algorithm,value="Playfair").pack()

tk.Label(root,text="Message").pack()
message=tk.Entry(root,width=40)
message.pack()

tk.Label(root,text="Key (Shift or Playfair Key)").pack()
key=tk.Entry(root,width=30)
key.pack()

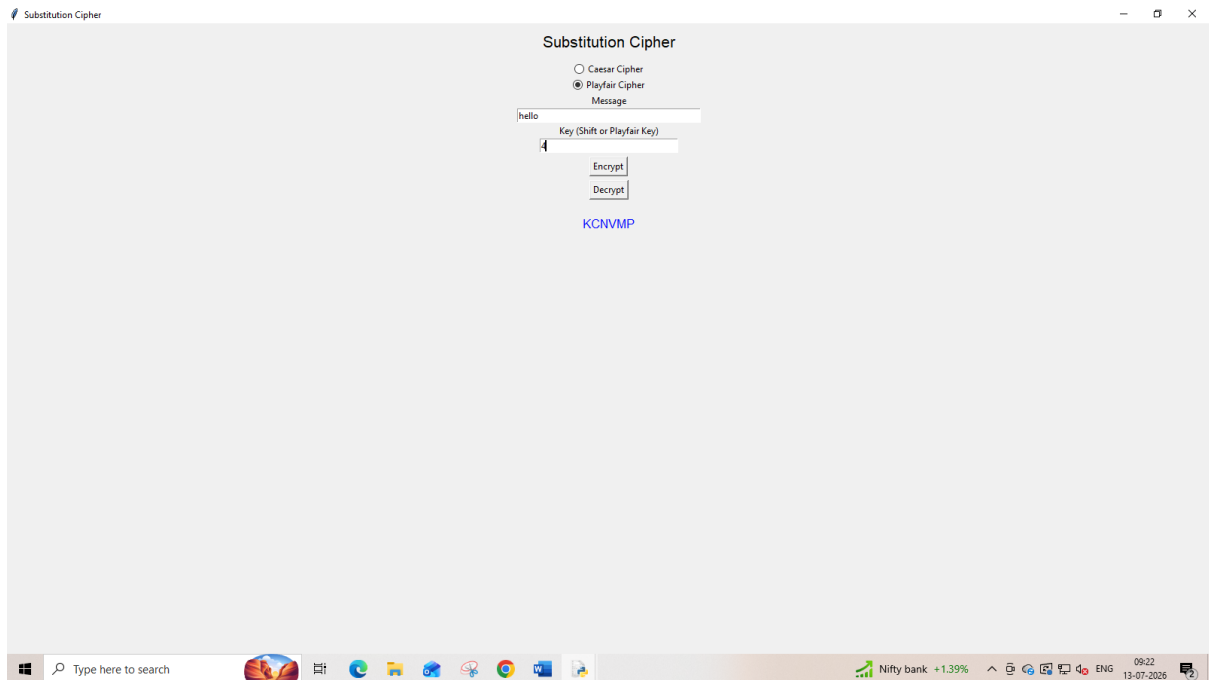
tk.Button(root,text="Encrypt",command=encrypt).pack(pady=5)
tk.Button(root,text="Decrypt",command=decrypt).pack()
```

```
output=tk.Label(root,text="",fg="blue",font=("Arial",12))
```

```
output.pack(pady=20)
```

```
root.mainloop()
```

Output:-



1.B) Design and implement algorithms to encrypt and decrypt messages using transposition techniques

CLI:-

```
# Practical 1B
```

```
# Rail Fence Cipher (Transposition Technique)
```

```
def encrypt(text, key):
```

```
    rail = [" " for _ in range(key)]
```

```
    row = 0
```

YASH BHAGAT
T072

```
direction = 1
```

```
for ch in text:
```

```
    rail[row] += ch
```

```
    row += direction
```

```
    if row == 0 or row == key - 1:
```

```
        direction *= -1
```

```
return ''.join(rail)
```

```
def decrypt(cipher, key):
```

```
    pattern = [[' ' for _ in range(len(cipher))] for _ in range(key)]
```

```
    row = 0
```

```
    direction = 1
```

```
    for col in range(len(cipher)):
```

```
        pattern[row][col] = '*'
```

```
        row += direction
```

```
        if row == 0 or row == key - 1:
```

```
            direction *= -1
```

```
index = 0
for i in range(key):
    for j in range(len(cipher)):
        if pattern[i][j] == '*':
            pattern[i][j] = cipher[index]
            index += 1
```

```
result = ""
row = 0
direction = 1
```

```
for col in range(len(cipher)):
    result += pattern[row][col]

    row += direction

    if row == 0 or row == key - 1:
        direction *= -1
```

```
return result
```

```
while True:
    print("\n===== Rail Fence Cipher =====")
```


Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

```
print("1. Encrypt")
print("2. Decrypt")
print("3. Exit")

choice = input("Enter Choice: ")

if choice == "1":
    text = input("Enter Plain Text: ")
    key = int(input("Enter Number of Rails: "))
    print("Encrypted Text:", encrypt(text, key))

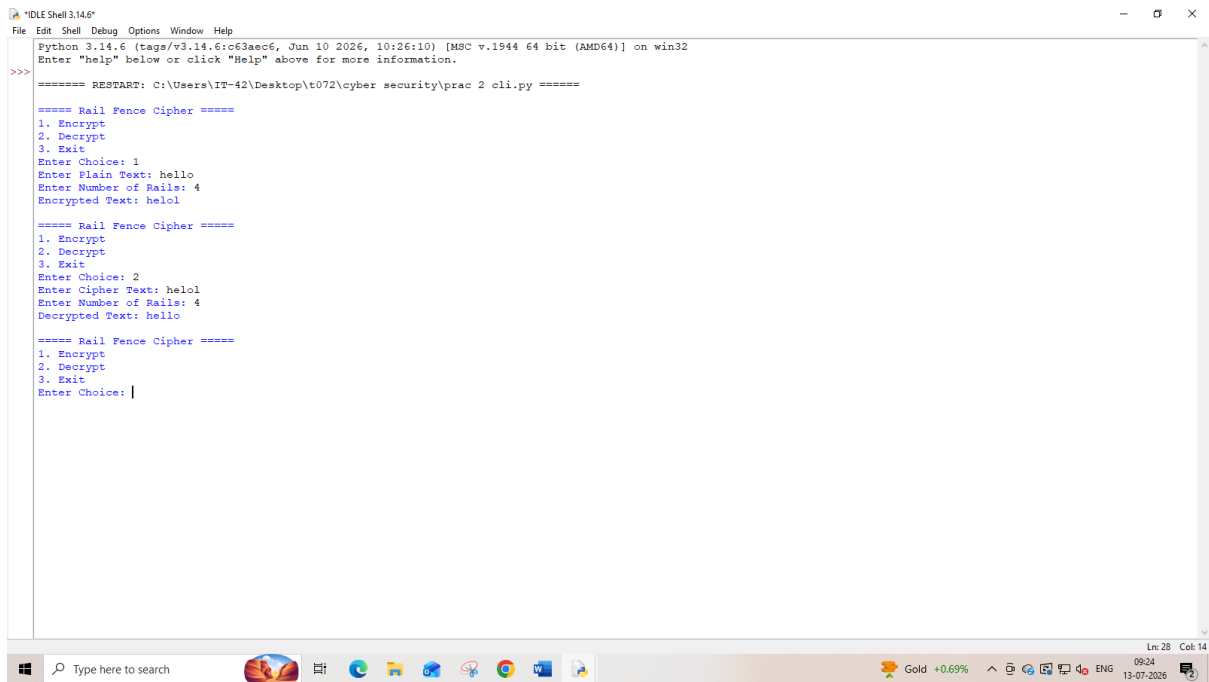
elif choice == "2":
    text = input("Enter Cipher Text: ")
    key = int(input("Enter Number of Rails: "))
    print("Decrypted Text:", decrypt(text, key))

elif choice == "3":
    print("Program Ended")
    break

else:
    print("Invalid Choice")
```

Output:-

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce



```
"IDLE Shell 3.14.6"
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:63aee6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\IT-42\Desktop\t072\cyber security\prac 2 cli.py =====

===== Rail Fence Cipher =====
1. Encrypt
2. Decrypt
3. Exit
Enter Choice: 1
Enter Plain Text: hello
Enter Number of Rails: 4
Encrypted Text: helol

===== Rail Fence Cipher =====
1. Encrypt
2. Decrypt
3. Exit
Enter Choice: 2
Enter Cipher Text: helol
Enter Number of Rails: 4
Decrypted Text: hello

===== Rail Fence Cipher =====
1. Encrypt
2. Decrypt
3. Exit
Enter Choice: |
```

GUI:-

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
# ----- Rail Fence Encryption -----
```

```
def encrypt(text, key):
```

```
    rail = [" " for _ in range(key)]
```

```
    row = 0
```

```
    direction = 1
```

```
    for ch in text:
```

```
        rail[row] += ch
```

```
        row += direction
```

YASH BHAGAT
T072

```
if row == 0 or row == key - 1:
```

```
    direction *= -1
```

```
return "".join(rail)
```

```
# ----- Rail Fence Decryption -----
```

```
def decrypt(cipher, key):
```

```
    pattern = [[' ' for _ in range(len(cipher))] for _ in range(key)]
```

```
    row = 0
```

```
    direction = 1
```

```
    # Mark the zig-zag pattern
```

```
    for col in range(len(cipher)):
```

```
        pattern[row][col] = '*'
```

```
        row += direction
```

```
    if row == 0 or row == key - 1:
```

```
        direction *= -1
```

```
    # Fill the pattern
```

```
    index = 0
```

```
    for i in range(key):
```

```
        for j in range(len(cipher)):
```

```
        if pattern[i][j] == '*':
            pattern[i][j] = cipher[index]
            index += 1

# Read plaintext
result = ""
row = 0
direction = 1

for col in range(len(cipher)):
    result += pattern[row][col]
    row += direction

    if row == 0 or row == key - 1:
        direction *= -1

return result

# ----- GUI Functions -----

def encrypt_message():
    try:
        text = message_entry.get()
        key = int(key_entry.get())

        if key < 2:
```

```
messagebox.showerror("Error", "Rails must be at least 2")
```

```
return
```

```
result = encrypt(text, key)
```

```
output_label.config(text="Encrypted: " + result)
```

```
except:
```

```
messagebox.showerror("Error", "Enter a valid number of rails")
```

```
def decrypt_message():
```

```
try:
```

```
text = message_entry.get()
```

```
key = int(key_entry.get())
```

```
if key < 2:
```

```
messagebox.showerror("Error", "Rails must be at least 2")
```

```
return
```

```
result = decrypt(text, key)
```

```
output_label.config(text="Decrypted: " + result)
```

```
except:
```

```
messagebox.showerror("Error", "Enter a valid number of rails")
```

```
# ----- Main Window -----
```

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

```
root = tk.Tk()
root.title("Rail Fence Cipher")
root.geometry("450x320")
root.resizable(False, False)

title = tk.Label(root, text="Rail Fence Cipher", font=("Arial", 16, "bold"))
title.pack(pady=10)

tk.Label(root, text="Enter Message").pack()

message_entry = tk.Entry(root, width=45)
message_entry.pack(pady=5)

tk.Label(root, text="Enter Number of Rails").pack()

key_entry = tk.Entry(root, width=10)
key_entry.pack(pady=5)

encrypt_btn = tk.Button(root, text="Encrypt", width=15, command=encrypt_message)
encrypt_btn.pack(pady=5)

decrypt_btn = tk.Button(root, text="Decrypt", width=15, command=decrypt_message)
decrypt_btn.pack(pady=5)

output_label = tk.Label(root, text="", font=("Arial", 12), fg="blue")
output_label.pack(pady=20)
```

root.mainloop()

Output:-

